

Microsoft Windows FileCrypt Driver: Complete Reverse Engineering Deep Dive

Total reverse engineering and rebuild of an obscure encryption minifilter driver

Research by: Ron Gladish, Noam Gordon, Eran Yeruhamie

As part of the final year of our B.Sc. in Computer Science at the College of Management Academic Studies, we had to choose a “*specialization*” - a program in which students get to choose more practice-oriented courses and do work that is a bit closer to the real tech industry.

The three of us, seeking a challenge and to learn and experience new things that we don't get to do in our jobs (each one at their own), chose Cyber Security out of the available options as our specialization.

In addition to the courses of each specialization, all students take a year-long ‘Final Project’ course, in which they are divided into groups and tasked with doing a project in the field of their specialty for the entire final year.

The three of us wanted to do some sort of research and/or development at a lower software level, whether it involved some kind of exploitation or reverse engineering. Fast forward to the day of the *Project Ideas Presentation*, our then-idea, which involved video game exploitation, was fairly brutally rejected (and rightfully so - there wasn't anything unique about it, and it added very little value). We were left without the slightest idea of what to do, and now we are almost behind schedule for submission. At the end of a very long brainstorming weekend, one of us, while going through literally every single official Microsoft driver on his personal Windows machine, encountered a driver named *FileCrypt*. After a googling (and a bit of AI prompting) session, only very basic and generic metadata information (The stuff you can find by clicking on ‘Properties’ on the .sys file), alongside users' troubleshooting BSOD issues, was available online — no official Microsoft documentation, no explicit mentions in articles, practically zero information.

This, together with the facts that:

- a. The driver was only 94 KB in size, not very big, and suitable for reverse engineering in the time we have.
- b. From its file description, which reads: “Windows sandboxing and encryption filter”, the driver had something to do with encryption - very fitting for our project requirement.

After discussing the matter between the three of us and tossing the thing into Ghidra to get a very basic first assessment of the magnitude of the situation, we came up with our new project's goal: Reverse engineer this driver to the fullest extent to figure out what it does and why it exists on our machines.

We couldn't set our goal to be “exploitation of the driver” because there is no guarantee that there are or aren't vulnerabilities in this driver; the project must have a concrete end goal that can be achieved within the timeframe of the academic year. Plus, the fact that we have no experience in reverse engineering and absolutely none in exploitation at even normal

user-mode applications, let alone at the kernel level, we wanted to avoid these kinds of goals entirely.

After formulating the idea, we contacted our supervisor, Dr. Gal Badishi, who very generously (and quite unexpectedly to us), agreed to have a video call with the three of us at the end of the weekend to give us a chance to bounce back with a fresh idea and make the schedule on the assignment of the project's idea.

In the call, Dr. Badishi had said that he liked our idea and gave the following circumstances:

- No prior reverse engineering experience
- A driver with absolutely no documentation or information about online
- An end result that is an open-source project that can potentially benefit other people

Gave us a green light.

Now, after what we are sure is an extremely interesting story for everyone, and without further delay, let's get into the technical details.

When we found the driver and honed in on our idea, we started reading driver documentation and articles on the [official Microsoft documentation](#). We did this to gain a better understanding of what we would see inside the driver's code, and because at one point, we wanted to write a driver ourselves to get a glimpse of how the development process for such a thing works. We very quickly realized that we need to understand what we need to focus on because, to absolutely no one's surprise, the documentation is massive.

Before going into the driver's decompiled code, we played around with Visual Studio's driver template projects, specifically "Kernel Mode Driver (KMDF)" to get to know the basic data types we are going to be working with, jumping around different header files, getting a general idea of how source code for Windows drivers look and trying to relate it all to the documentation we were reading at the same time.

After a bit of that, we set up [Ghidra](#) with the driver's PDB files, which, fortunately, were made public by Microsoft. So now, into the code!

Not quite. We are a team of three, and we must be able to work in parallel. With regular git, it would have been very simple: create a branch, commit to it, and merge. Unfortunately, Ghidra does not integrate well with Git because it creates files that are difficult to merge and practically unusable with a git environment. However, the NSA has devised a solution. Ghidra comes with a runnable service called the Ghidra Server, which allows multiple people working on the same reversing project to synchronize their work, like how Git works. Unlike git, Ghidra Server does not have an equivalent to GitHub, so we had to host our own instance of Ghidra Server.

Due to our limited experience in this type of work, we were unsure of the optimal way to divide the tasks between us. What we did know is that there are some things we can do independently of each other, so that none of us blocks the progress of the others. To start, we divided our work into the following:

- One person begins reverse engineering the driver's code in Ghidra
- Another begins research about methods to debug the driver in a VM. *More on this at the end*

- The last continues general driver research and debug methods, and then joins the second person in the debugging efforts.

The plan was that the person working on the driver's code would handle the starting point of the driver and, after learning more about its actual functionality and code paths, would be able to form a more detailed plan going forward.

Let us begin with the driver's code: We then began examining the driver's decompiled code, starting with the `DriverEntry`.

At this point, we wanted to see how what we read in the docs, combined with what we see in the driver template project, compares with the decompiled driver code.

Because this is not a device driver, at first glance, we did not find what we were looking for; however, we did encounter filter-driver functions, such as `FltRegisterFilter`, `FltStartFiltering`, and `FltUnregisterFilter`. We now knew that what we were looking at was a [minifilter driver](#). We later confirmed this by examining what we revealed to be the `FilterRegistration` object more closely. This sent us back to the docs to read all about filter drivers - what they are, how they work, how they interact with the rest of the system, etc. While reading Microsoft's documentation, we came across a [GitHub repository](#) that contains a variety of example drivers, including a comprehensive set of minifilter drivers. [This driver](#), named Scanner, caught our attention in particular because, judging from its description, it performs a task roughly in the same area as what our driver does.

This repo turned out to be very useful for referencing Windows API calls and how they look in actual source code.

Back to Ghidra, we wanted to start renaming and retyping the most obvious variables and function parameters, starting, for example, with the arguments of the `DriverEntry`.

```
void DriverEntry(  
    ushort* param_1,  
    ushort* param_2  
)
```

Immediately, we encountered our first issue: our Ghidra is missing the header files from the [Windows Driver Development Kit](#). Luckily, we found [this article](#), in which the author lays out the solution to what seemed like a plug and play feature in Ghidra - it wasn't.

After going through it step by step, we realized we were quite fortunate because this article was published just 2 months before this point.

The author made use of [CyberChef](#) - we had to expand his "recipe" just a bit, the last link should open the app with our recipe already loaded. We also have it in our [GitHub](#).

```
DriverEntry(  
    PDRIVER_OBJECT PDriverObject,  
    PUNICODE_STRING PRegistryPath  
)
```

And voilà! Throughout the project, we had to regenerate our header files several times to

include additional headers, as we encountered rarer code that utilized very specific modules.

**Note - from now on, the code in the snippets we will be showing is from a recreation project of the driver's code, which converts the decompiled Ghidra code into actual buildable C code. More on this is available at the end of the article.*

Now, back to the DriverEntry. The first thing it does is change global values for memory management based on the Windows version the driver is running on:

```
NTSTATUS osVersionStatus;
OSVERSIONINFOW lpVersionInfo;
...

memset(&lpVersionInfo.dwMajorVersion, 0, 0x110);
lpVersionInfo.dwOSVersionInfoSize = 0x114;

osVersionStatus = RtlGetVersion(&lpVersionInfo);

if (-1 < osVersionStatus &&
    (
        6 < lpVersionInfo.dwMajorVersion ||
        lpVersionInfo.dwMajorVersion == 6 && 1 < lpVersionInfo.dwMinorVersion
    )
)
{
    ExDefaultNonPagedPoolType = NonPagedPoolNx;
    ExDefaultMdlProtection = MdlMappingNoExecute;
}
```

The versioning scheme in use can be found [here](#). Essentially, this code checks if the version of Windows is higher than Windows 7. This check would include all of the following Windows versions:

Windows 11	10.0
Windows Server 2022	10.0
Windows Server 2019	10.0
Windows 10	10.0
Windows Server 2016	10.0
Windows Server 2012 R2	6.3
Windows 8.1	6.3
Windows 8	6.2
Windows Server 2012	6.2

Seems like someone has been forgetting to update this number in the last 10 years.

If we are using a later version of Windows, the driver enables two memory protections: it marks memory pages as non-executable and makes MDL (Memory Descriptor List) mappings default to non-executable. This is presumably because this feature is not

supported in earlier versions of Windows.

Next, the driver registers its event provider for event tracing by calling `EtwRegister`. The driver then reads from its provided registry path the following keys:

`DebugTraceLevel`, `EncryptMedia`, `EncryptAll`, `BypassAccessChecks`, `FilterEmulatedExternalDrive`.

For more information about these keys, their values, and all other registry paths the driver accesses, you can refer to our technical document of the driver, which is available on our GitHub. The driver stores these values in memory and will later make decisions regarding device attachment, access checks, and encryption, based on these flags. The driver will access many other registry paths throughout its operation.

After initializing a couple of lookaside lists for operations that happen later on, the driver does something particularly interesting:

```
isMobileOS = FsRtlIsMobileOS();  
if (isMobileOS == TRUE)  
{  
    FilterRegistration.OperationRegistration = CallbacksMobile;  
}
```

It registers different callbacks depending on the return value of `FsRtlIsMobileOS`. This is an undocumented function from `ntoskrnl` that is also exported in `ntifs.h`.

On a “non-mobile” OS, the driver registers only Pre+Post Create (`IRP_MJ_CREATE`) callbacks. In contrast, on a “mobile” OS, the driver registers additional Pre+Post Read (`IRP_MJ_READ`) and Write (`IRP_MJ_WRITE`) callbacks.

This single function has puzzled us for the majority of the project, not because of its complexity or inconsistency, but because of its implications for the overall purpose of this driver. A couple of notable observations:

- From what we have gathered from unofficial sources and personal testing, this driver first appeared around 2015 in Windows 10. *This makes the decision to check for earlier versions of Windows at the start of the DriverEntry very questionable.*
- Windows on Mobile was discontinued in 2020.
- The driver is included in Windows 11 (Released in October 2021) and received a non-insignificant update in version 24H2 (Released in October 2024).
- The code of `FsRtlIsMobileOS`, which we will show in a moment, does not explicitly check for anything that we were able to link directly to the Windows Mobile OS.

Why does an “Error-Control - Critical” and non-stopable driver, that is shipped with the latest version of Windows, care about a dead technology at such a high level?

The code of `FsRtlIsMobileOS`, as seen in a decompiled `ntoskrnl.exe`, did not provide profound insight:

```

BOOLEAN FsRtlIsMobileOS(VOID)
{
    NTSTATUS status;
    PVOID ptr = NULL;

    status = TmCurrentTransaction(&ptr);

    return status == STATUS_NOT_SUPPORTED;
}

```

TmCurrentTransaction is not a publicly exported routine, but it is presumably part of the [Windows Transaction Manager](#). We haven't been able to go deeper than ntoskrnl.exe and into TmCurrentTransaction because it led to what appeared to be an API stub DLL.

```

*****
* POINTER to EXTERNAL FUNC... *
*****
<DEFAULT>
undefined          AL:1          <RETURN>
45 TmCurrentTransaction <<not bound>>
_imp_TmCurrentTransaction
1401453c0          addr          EXT-MS-WIN-NTOS-TM-L1-1-0.DLL::TmCurrentTransaction

```

Consulting both the *Windows Internals 7th edition* and the official documentation, we were unable to confirm that the KTM (Kernel Transaction Manager) is not included in the Windows Phone OS. Most API documentation pages of KTM routines list Windows Vista as a minimum requirement for their usage, without explicit mention of any Windows Phone versions. At one point in the research, we theorized that this function might be related to Windows on ARM, considering the new Surface Snapdragon notebooks with the latest Windows Recall features that may require enhanced security and encryption. This theory was based on pure speculation, and we have no evidence to support it. We have no reason to believe that the Windows KTM is unavailable on ARM. In fact, it sounds silly. We did not debug Windows ARM VMs, but upon examining different registries, they did not appear to differ from our Intel-based machines and VMs.

Ultimately, we came to terms with the fact that this function likely does what it claims: checks if we are running on a Mobile OS, specifically and most likely, a Windows Phone.

Upon a successful call to FltRegisterFilter, the driver then calls the StSecInitialize function. In it, it initializes, among other things:

- Global hashing objects using functions from bcrypt.h, including a SHA256 algorithm provider
- 3 global objects that cache both operations and data from the registry
 - A dynamic g_StSecCacheGenericTable for storing *Chamber keys*
 - 2 static lists, g_StSecSecurityDescriptorCacheList, g_StSecFolderPropertyCacheList, that are initialized once with data from the registry. The first is for storing Windows security descriptors, and the second is for storing *chamber* properties about specific paths.

Refer to the [technical overview](#) that was mentioned previously for more information about

these specific objects.

After initializing numerous global variables and performing a series of registry reads, the driver notifies the Filter Manager by calling `FltStartFiltering` that it is ready to begin attaching to volumes and filtering I/O requests.

Before exploring the callbacks, we must first review the *Instance Setup Callback* - `FCInstanceSetup`. This function determines whether the driver would like to attach an instance of itself to a given volume.

FCInstanceSetup

First thing the driver does is check the volume device and system file types that it received from the instance setup callback:

```
/* Only attach to file systems */
if (VolumeDeviceType != FILE_DEVICE_DISK_FILE_SYSTEM)
{
    goto FCInstanceSetup_cleanup_and_return;
}

/* Only attach to the following filesystems: RAW, NTFS, FAT, and EXFAT */
if (1 < VolumeFilesystemType + ~FLT_FSTYPE_RAW &&
    VolumeFilesystemType != FLT_FSTYPE_EXFAT)
{
    goto FCInstanceSetup_cleanup_and_return;
}
```

The driver only supports the following file systems: RAW, NTFS, FAT, and exFAT. Next, the driver checks for a particular device characteristic:

```
doesVolumeNotSupportRemovableMedia =
(volumeProperties->DeviceCharacteristics & FILE_REMOVABLE_MEDIA) == 0;
```

"Removable" means that the media of the device is removable. For example, CD-ROM drives or card readers for flash media. It then checks if one of the flags that it read in `DriverEntry` earlier is on, and if so, it checks if the label of the volume is equal to "SDCARD"

```
if ((gFCFlags & FilterEmulatedExternalDriveFlagBit) != 0)
{
    /* Check if the VolumeLabelLength is six wchars in length
     * (12/13 byte longs).
     * If it is, then compare the VolumeLabel with the string "SDCARD" */
    if ((fsVolumeInfo->VolumeLabelLength & 0xfffffffffe) == 12)
    {
        compResult = _wcsnicmp(
            fsVolumeInfo->VolumeLabel,
```

```

        L"SDCARD",
        fsVolumeInfo->VolumeLabelLength >> 1
    );
    isVolumeSdCard = compResult == 0;
}
}

```

These operations culminate in the following code segment, in which the driver does an additional probe on the type of OS it is running on:

```

isMobileOS = FsRtlIsMobileOS();
if (isMobileOS == FALSE)
{
    status = FCpRetrieveAppPairingId(FltObjects);
    if (status < 0)
    {
        FCInstanceSetup_sd_card_check:
        if (!isVolumeSdCard)
        {
            goto FCInstanceSetup_cleanup_and_return;
        }
    }
}
else if (doesVolumeNotSupportRemovableMedia)
{
    goto FCInstanceSetup_sd_card_check;
}

```

FCpRetrieveAppPairingId checks whether a specific file named “WPApSettings.dat” exists in the protected “System Volume Information” folder on the device.

On mobile, continue the attachment process only on SD cards.

For non-mobile devices, continue the attachment process if the special file exists **or** if the device is an SD card. The non-mobile environment has an extra check that may allow it to continue the attachment.

The check for WPApSettings.dat being unique to non-mobile Windows systems seemed very arbitrary to us, so we decided to dig deeper. Firstly, what does this file even contain? As usual in this project, googling or prompting an LLM did not yield any practical information. When searching for it on our personal machines’ drives and our VMs, we could not find it in any driver; however, we did find a similarly named file called “WPSettings.dat”. This looked like a promising lead, so we decided to investigate this file further. What we found were mostly forum posts complaining about Windows automatically generating this file in every file system that gets connected to the machine. Along with these, there were several different claims about which Windows mechanism generates this file; the most credible claim was that a service called StorSvc was responsible for it. So now all we have to do is hope that WPSettings and WPApSettings are related and that reversing the StorSvc service might shed some light on this matter.

StorSvc is actually a DLL file used by an instance of SvcHost.exe. Many of the Windows services are implemented as different DLL files hosted by an instance of SvcHost. Now, all we have to do is a bit of reversing and look for any references to WPAAppSettings or WPSettings. What we found was a huge service that we did not have the capacity to fully understand. However, we did find one thing that was very interesting - a function called "PairStorageCardForApps". The name of this function is too similar to "FCpRetrieveAppPairingID" to be a coincidence. When looking further into this function, we found two interesting things:

- This function creates a WPAAppSettings.dat file.
- This allows Filecrypt to attach to a volume using the FilterAttach function.

```
if ((uVar1 & 5) != 0) {
    lVar2 = GetDeviceAppPairingId(this_00,&local_58);
    if (lVar2 < 0) goto LAB_18002a2ca;
    lVar2 = SetStorageCardAppMetadata(this,param_1,param_2,&local_58);
    if (lVar2 < 0) goto LAB_18002a2ca;
}
p_Var4 = (_GUID *)0x0;
/* Found actual usage of the Filecrypt driver */
iVar3 = FilterAttach(L"Filecrypt",*(longlong *) (this + (longlong)(int)param_1 * 8 + 0x28) + 4 + (ulonglong)param_2 * 0x458);
```

But what logic leads to this? Inside the code, we found a vital clue:

In the function-call stack leading to the code above, StorSvc creates a folder named WindowsApps. A simple Google search revealed that this folder contains applications installed from the Microsoft Store. In the Windows settings, you can change the default installation folder for applications from the Microsoft Store. However, when we configure our installation location to be the C volume, the WpAppSettings file is not created. Why? Through trial and error, we discovered that when we set a flash drive as the default Microsoft Store installation folder, WpAppSettings.dat is created. More detailed information about this requires us to dig deeper into StorSvc, but with our time being constrained, we decided to focus our attention on our driver. For now, we are satisfied knowing that on non-mobile systems, we can attach a FileCrypt instance to a volume by connecting a flash drive and setting it as the default installation location for Windows applications.

Now, back to FCInstanceSetup, we left off on the following code block that searched for the app pairing Id:

```
isMobileOS = FsRtlIsMobileOS();
if (isMobileOS == FALSE)
{
    status = FCpRetrieveAppPairingId(FltObjects);
    if (status < 0)
    {
        FCInstanceSetup_sd_card_check:
        if (!isVolumeSdCard)
        {
            goto FCInstanceSetup_cleanup_and_return;
        }
    }
}
```

```

else if (doesVolumeNotSupportRemovableMedia)
{
    goto FCInstanceSetup_sd_card_check;
}

```

If one of the above conditions is true and the driver decides to continue, it then initializes a volume context for the instance (refer to the technical documentation - CUSTOM_FC_VOLUME_CONTEXT). In the volume context, the driver holds a BCrypt algorithm handle (BCRYPT_ALG_HANDLE) alongside the volume's sector size, a flag indicating whether encryption is enabled for it, another flag that affects security checks later on, and some more metadata about the volume. The encryption variables of the context are initialized inside FCpEncVolumeStart, and after this, the volume context is set on the volume using FltSetVolumeContext.

```

volumeContext->BcryptAlgHandle.BcryptAlgHandle = NULL;
volumeContext->BcryptAlgHandle.ObjectLength = 0;
volumeContext->BcryptAlgHandle.EncryptionSectorSize = volumeContext->SectorSize;
volumeContext->BcryptAlgHandle.SecretKeySize = 0x10;
volumeContext->EncryptionEnabled = 1;
encryptVolumeStartStatus = FCpEncVolumeStart(&volumeContext->BcryptAlgHandle);

...

FltSetVolumeContext(
    FltObjects->Volume,
    FLT_SET_CONTEXT_KEEP_IF_EXISTS,
    volumeContext,
    NULL
);

```

We then have another call to FsRtlIsMobileOS (Why not reuse the same value from before?).

```

vqb = diskDeviceObject->Vpb;
isMobileOS = FsRtlIsMobileOS();
if (
    isMobileOS == FALSE &&
    (vqb->DeviceObject->Flags & DO_SUPPORTS_PERSISTENT_ACLS) != 0
)
{
    volumeContext->VerificationNeeded = FALSE;
}
else
{
    volumeContext->VerificationNeeded = TRUE;
    vqb_deviceObject = vqb->DeviceObject;
    vqb_deviceObject->Flags = vqb_deviceObject->Flags |
    DO_SUPPORTS_PERSISTENT_ACLS;
}

```

And in the if-else, the driver determines whether the volume requires security verification:

- Non-mobile systems without ACL support also require verification
- Mobile systems always require verification
 - The DO_SUPPORTS_PERSISTENT_ACLS flag is added to the device

The VerificationNeeded flag will later be used in the pre-operation of the create callback.

In the block, if the volume does not support removable media but the FilterEmulatedExternalDrive flag is set, the driver forces all device objects in the driver stack to have the FILE_REMOVABLE_MEDIA characteristic:

```
if (
    (gFCFlags & FilterEmulatedExternalDriveFlagBit) != 0 &&
    doesVolumeNotSupportRemovableMedia
)
{
    highestDeviceObject = IoGetAttachedDeviceReference(diskDeviceObject);

    while (highestDeviceObject != NULL)
    {
        highestDeviceObject->Characteristics =
            highestDeviceObject->Characteristics | FILE_REMOVABLE_MEDIA;
        lowerDeviceObject = IoGetLowerDeviceObject(highestDeviceObject);
        ObfDereferenceObject(highestDeviceObject);
        highestDeviceObject = lowerDeviceObject;
    }
}
```

And as a closing note in the instance setup, if the BypassAccessChecks flag is set, the driver strips the device object of the FILE_DEVICE_SECURE_OPEN characteristic:

```
if ((gFCFlags & BypassAccessChecksFlagBit) != 0)
{
    diskDeviceObject->Characteristics =
        diskDeviceObject->Characteristics & ~FILE_DEVICE_SECURE_OPEN;
}
```

To summarize the instance setup:

- The driver only attaches to RAW, NTFS, FAT, and ExFAT disk file systems
- On non-mobile systems, the driver attempts to retrieve an "App Pairing ID" first. If that fails, it only attaches to SD card volumes labeled "SDCARD"
- On mobile systems, it only attaches to SD cards, not internal storage
- **The check for SD cards only happens if the FilterEmulatedExternalDrive flag is set**
- Initializes the volume context of the driver (CUSTOM_FC_VOLUME_CONTEXT)
 - DeviceName: DOS name of the volume (e.g., "C:")
 - SectorSize: Disk sector size (defaults to 512 bytes, uses larger if available)
 - BcryptAlgHandle: Encryption settings for AES volume-level encryption
 - EncryptionEnabled: Boolean flag indicating if encryption is active
 - VerificationNeeded: Boolean flag for security verification requirements
- Calls FCpEncVolumeStart to initialize a BCrypt AES algorithm provider
- On mobile or volumes without persistent ACLs support, the driver forces the

DO_SUPPORTS_PERSISTENT_ACLS flag

- If the FilterEmulatedExternalDrive flag is set, the driver applies the FILE_REMOVABLE_MEDIA characteristic to the entire device stack
- If the BypassAccessChecks flag is set, the driver removes the FILE_DEVICE_SECURE_OPEN characteristic from the volume

In the next part, we will delve into the largest callback of the driver by far, and the only one registered on non-mobile systems: *IRP_MJ_CREATE* - *FCPreCreate* & *FCPostCreate*.

IRP_MJ_CREATE - FCPreCreate & FCPostCreate

Every kernel-mode driver must handle IRP_MJ_CREATE requests in a DRIVER_DISPATCH callback function. This is why the callback must be registered on both mobile and non-mobile systems. This callback is triggered by different user operations that can all be equated to:

- Creating a file (or directory)
- Opening a file (or directory)

Although registered on both, the functionality of this callback differs significantly depending on whether it is running on a mobile or non-mobile system.

FCPreCreate

When FltMgr receives the I/O operation, it calls the minifilter *preoperation* callback routines in order of altitude from highest to lowest. FltMgr then forwards the I/O request to the next-lower driver for further processing.

When FltMgr receives the I/O request for *completion*, it calls each minifilter driver's *postoperation* callback routines in reverse order, from lowest to highest.

We first retrieve the volume context that we set up earlier during the instance setup. If this fails, the callback returns FLT_PREOP_COMPLETE. In this case, the filter manager won't send the I/O operation to any minifilter drivers below the caller in the driver stack or to the file system. The filter manager only calls the post-operation callback routines of the minifilter drivers above the caller in the driver stack. The next step the driver takes is to retrieve and parse the file's name, after which we have the *fullPathBuffer* and *fileNameLength* variables. And at this point, we are introduced to the concept of *chambers*:

```
UNICODE_STRING chamberPath = {0, 0, NULL};
CUSTOM_FC_CHAMBER_DATA chamberData;

...

chamberPath.Length = fileNameLength - 2;
chamberPath.MaximumLength = fileNameLength;
chamberPath.Buffer = fullPathBuffer;

...
```

```

chamberData.ChamberId = NULL;
chamberData.ChamberType = 0;
chamberData.InputPath = &chamberPath;
chamberData.SecurityDescriptor = NULL;

```

The majority of the code going forward will almost always be based on this concept; therefore, for now, a brief explanation of what chambers are: Chambers are essentially groupings of files based on their location, purpose, or security requirements. Each chamber has its own unique encryption keys, derived from a master key, which provides cryptographic isolation between different types of content. As we will soon see, a chamber can be a folder, a specific Windows App, or something else. In the code above, we see that initially, the chamber's path is set to the file we just accessed. After initializing the path of the chamber, the driver retrieves the type of the chamber (which determines the encryption key to be used) and its security descriptor for access checks. This is accomplished in 2 ways, the other being the fallback to the first:

```

UNICODE_STRING chamberPath = {0, 0, NULL};
CUSTOM_FC_CHAMBER_DATA chamberData;
PWCHAR chamberIdStr = NULL;
UNICODE_STRING chamberPath = {0, 0, NULL};
PFLT_FILE_NAME_INFORMATION fileNameInfo = NULL;
PNPAGED_LOOKASIDE_LIST securityDescriptor = NULL;
ACCESS_MASK accessMask = 0;

...

kernelStackStatus = KeExpandKernelStackAndCalloutEx(
    FcObtainSecurityInfoCallout,
    &chamberData,
    0x3000,
    0,
    0
);

if (kernelStackStatus < 0)
{
    ...
    chamberData.Status = StSecGetSecurityDescriptor(
        &chamberPath,
        securityDescriptor,
        &chamberIdStr,
        &accessMask
    );
}
/* Do mostly the same thing FcObtainSecurityInfoCallout does, but inline
right here*/
...

```

The driver calls FcObtainSecurityInfoCallout via KeExpandKernelStackAndCalloutEx. The KeExpandKernelStackAndCalloutEx function

is a safety mechanism designed to overcome the fundamental limitations of fixed-size kernel stacks. This function provides guaranteed stack space for kernel operations that might otherwise trigger a stack overflow. The driver first attempts to call `FCpObtainSecurityInfoCallout` (which internally calls `StSecGetSecurityDescriptor`) with an expanded stack to ensure there is sufficient space for the complex security descriptor processing that occurs in the latter. If stack expansion fails, the driver falls back to calling `StSecGetSecurityDescriptor` directly with the current (limited) stack. `StSecGetSecurityDescriptor` is where we begin to delve into the security mechanisms of the driver.

If the stack expansion doesn't fail, `FCpObtainSecurityInfoCallout` is called with the safety of an expanded kernel stack. It takes a file path and determines which encryption chamber it belongs to, as well as the security policies that should govern access to it.

After validating that the input path isn't empty, the function immediately delegates the heavy lifting to `StSecGetSecurityDescriptor`.

```
VOID
FCpObtainSecurityInfoCallout(
    PCUSTOM_FC_CHAMBER_DATA ChamberData
)
{
    BOOLEAN chamberMatch;
    NTSTATUS status;
    PWCHAR assignedChamberId;
    PSECURITY_DESCRIPTOR securityDescriptor = &ChamberData->SecurityDescriptor;
    PWCHAR* chamberId;
    PCUNICODE_STRING chamberPath = ChamberData->InputPath;

    ChamberData->ChamberId = NULL;

    if (chamberPath->Length == 0)
    {
        ChamberData->Status = 0;

        return;
    }

    chamberId = &ChamberData->ChamberId;
    status = StSecGetSecurityDescriptor(chamberPath, securityDescriptor,
    chamberId, &ChamberData->ChamberType);

    ...
}
```

This is where FileCrypt's path-to-chamber mapping occurs. The function searches through cached security descriptor policies loaded from the registry during driver initialization, looking for patterns that match the current file path. At this point, we should explain what a Windows Security Descriptor is:

Windows Security Descriptors form the foundation of access control throughout the Windows operating system. They are binary data structures that define who can access objects and what actions they can perform on them. Security Descriptors contain four core components

(Owner SID, Group SID, DACL, and SACL) that work together to provide access control and auditing capabilities. Security Descriptors work with any securable Windows object, including files and directories (NTFS only), registry keys, processes and threads, mutexes, and other synchronization objects, as well as Active Directory objects.

This function serves as the entry point to a complex security logic, which calls numerous functions that extend several levels deep in the driver's call stack. A quick summary of it before we get into the weeds:

It first calls `StSecpGetStorageFolderStringSecurityDescriptor` to get a string-format security descriptor that matches the input path. This function:

- Searches through a cached list of security descriptor policies (One of the caches set up by `StSecInitialize`, which `DriverEntry` called)
- Finds a policy element whose path pattern matches the input path
- Processes any parameter substitutions in the security descriptor template (like `<PackageFamilyName>`)
- Returns a fully resolved security descriptor string

It then converts the security descriptor string to binary by calling the `SeConvertStringSecurityDescriptorToSecurityDescriptor` function.

It then calls `StSecpFindFolderPropertyPolicyElement`, which searches through another cache (folder property cache) to find additional metadata for the path:

- Returns a policy element containing `ChamberId` and `FolderId`
- Uses path pattern matching similar to the security descriptor lookup

After this, `StSecGetSecurityDescriptor` handles two types of chamber IDs:

1. Static Chamber IDs: Direct string values that are simply copied to the output
2. Parameterized Chamber IDs: Enclosed in `<>` brackets, these require resolution:
 - `<PackageFamilyName>` or `<ProductId>`: Uses the path component directly
 - `<PackageFullName>`: Calls `StSecpPackageFamilyNameFromFullName` to extract the family name

And at the end, `StSecpPackageFamilyNameFromFullName` is called to parse Windows Store app package names:

- Full name format:
`PublisherName.AppName_Version_Architecture_ResourceID_PublisherID`
- Extracts: `PublisherName.AppName_PublisherID` (the family name)

So that was a mouthful. Let's see this mess play out in the code:

```

NTSTATUS
StSecGetSecurityDescriptor(
    PCUNICODE_STRING InputPath,
    PSECURITY_DESCRIPTOR OutSecurityDescriptor,
    PWCHAR* OutChamberId,
    PULONG OutChamberType
)
{
    NTSTATUS return_status;
    int status;
    PCUSTOM_FC_STSEC_FOLDER_PROP_CACHE_LIST_ENTRY policyElement;
    PWCHAR stringBuffer;
    longlong stringLength;
    PWCHAR chamberId = NULL;
    PWCHAR securityDescString = NULL;
    UNICODE_STRING paramName;
    UNICODE_STRING componentName;
    UNICODE_STRING policyComponent;
    UNICODE_STRING remainingPath;
    UNICODE_STRING pathSuffix;
    PWCHAR chamberIdStr = NULL;
    longlong stringSegmentLength;
    UNICODE_STRING tempString;

    chamberId = chamberIdStr;

    if (
        InputPath == NULL ||
        InputPath->Buffer == NULL ||
        OutSecurityDescriptor == NULL ||
        OutChamberId == NULL ||
        OutChamberType == NULL
    )
    {
        StSecGetSecurityDescriptor_invalid_parameter:
        return_status = STATUS_INVALID_PARAMETER;
        goto StSecGetSecurityDescriptor_cleanup_and_return;
    }

    return_status =
    StSecpGetStorageFolderStringSecurityDescriptor(InputPath,
    &securityDescString);

    ...

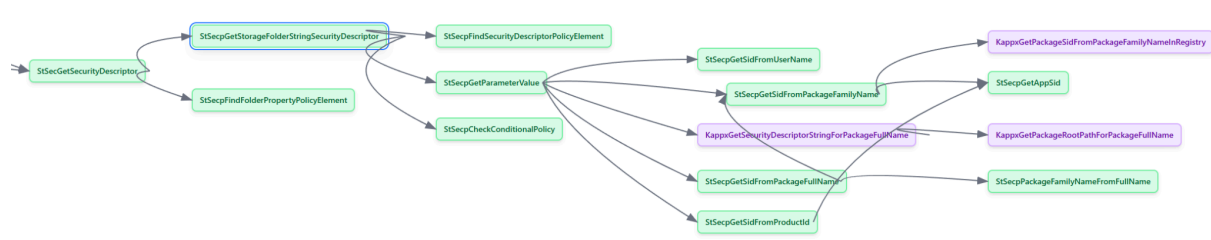
```

StSecpGetStorageFolderStringSecurityDescriptor transforms template security descriptors into fully resolved, ready-to-use security descriptor strings by performing parameter substitution.


```
NTSTATUS
StSecpGetStorageFolderStringSecurityDescriptor(
    PCUNICODE_STRING FolderPath,
    PWCHAR* OutStringSecurityDescriptor
)

```

Callstack overview:



The function implements a template-based security descriptor system where Security policies are stored in the registry with parameter placeholders. Those parameters are resolved based on the actual file path being accessed, and the final SDDL string is constructed with all parameters substituted.

StSecpFindSecurityDescriptorPolicyElement searches the cached security descriptor policies (loaded from the [registry](#)) to find a matching entry. The cache contains entries of type CUSTOM_FC_STSEC_SEC_DESC_CACHE_LIST_ENTRY:

- Path: Registry path pattern (may contain parameters like <PackageFamilyName>)
- SecurityDescriptor: SDDL template string
- DebugValue: Additional security descriptor for debug profiles

Then StSecGetSecurityDescriptor walks through the input paths segment by segment to find if there is a parameter in the path, for example: C:\Users\<User>\AppData. If a parameterized variable is found, it handles different parameter types through StSecpGetParameterValue:

StSecpGetStorageFolderStringSecurityDescriptor

```

if (*parameterName.Buffer == L'<' &&
    parameterName.Buffer[(ulonglong)(parameterName.Length >> 1) - 1] ==
    L'>')
{
    if (sid.Buffer == NULL || parameterCount == 2)
    {
        return_status = STATUS_UNSUCCESSFUL;
        goto
StSecpGetStorageFolderStringSecurityDescriptor_cleanup_and_return;
    }

    return_status = StSecpGetParameterValue(
        &parameterName,
        &sid,

```

```
(PWCHAR*)(&parameters + maxStringLength)
);
```

StSepcGetParameterValue converts the extracted value based on the parameter type:

- <User>: Converts username to SID
- <PackageFamilyName>: Converts to app-specific SID
- <PackageFullName>: Extracts family name, then converts to SID
- <ProductId>: Converts Product identifier to SID

A [Windows Security Identifier](#) (SID) is a unique digital "fingerprint" that Windows uses to identify and distinguish different security principals in the system. Examples include a user account, a computer account, or a thread or process that runs in the security context of a user or computer account. A SID is a variable-length binary value that gets represented as a readable string when you see it displayed. The string format looks like this:

S-1-5-21-1234567890-987654321-1122334455-1001.

The driver is particularly focused on app container SIDs, which Windows uses for sandboxed applications, such as those from the Microsoft Store. At this point, we should take a step back and explain the PackageFamilyName, ProductId, and PackageFullName business. Let's review the Windows App naming hierarchy and what each of those identifiers represents:

PackageFullName is the most complete identifier for a Windows App installation. It contains all the information needed to uniquely identify a specific version and variant of an app. The format follows this pattern:

PublisherName.AppName_Version_Architecture_ResourceID_PublisherID

For example:

Microsoft.Office.Word_16.0.14228.20216_x64_en-us_8wekyb3d8bbwe

This full name is crucial because it identifies the exact installation—the specific version, architecture, and language variant of an app installed on the system.

The **PackageFamilyName** is a simplified identifier that groups all versions and variants of an app. It follows this format:

PublisherName.AppName_PublisherID

Using our example above, the family name would be:

Microsoft.Office.Word_8wekyb3d8bbwe

This identifier is essential because different app versions don't change it, and it's architecture-independent.

The **ProductId** is a more abstract identifier that can take multiple forms:

- Traditional GUID format: {CF8E2E00-F4B3-11E3-AC10-0800200C9A66}
- Microsoft Store ID: 9WZDNCRFJ364 (typically 12 characters)

As we saw previously, the driver uses a concept called "chambers" - isolated encryption contexts that segregate data. Each Windows app can have its own encryption chamber. This chamber system ensures that each app's data is encrypted with unique keys, and apps cannot access each other's encrypted data even if they become aware of their own encryption keys.

Security descriptors in the registry? Parameterized registry paths? Why though?

In typical desktop environments, this allows system administrators (and potentially regular users) to define security policies with placeholders that are filled in at runtime. A couple of scenarios that demonstrate the usage of such a system:

1. User-Centric Policies

Parameter: <User>

Value: JohnDoe

→ Encrypt all files for a specific user, regardless of which app they use

With a <User> parameter, the system admin can create user-specific encryption chambers. This can be used for machines that are shared between different users, where you want to protect a user's data regardless of which app created it.

2. App Family Policies

Parameter: <PackageFamilyName>

Value: Microsoft.Office.Word_8wekyb3d8bbwe

→ Encrypt all Word documents, regardless of Word's version

This is the most common scenario. When Word gets updated from version 16.0 to 16.1, the PackageFamilyName stays the same, so encrypted documents remain accessible after the update.

3. Version-Specific Policies

Parameter: <PackageFullName>

Value: Microsoft.Office.Word_16.0.14228.20216_x64_en-us_8wekyb3d8bbwe

→ Apply special encryption only to a specific version

Sometimes, you need policies that target a specific version of an app, perhaps due to a potential security concern related to a particular app version. The PackageFullName includes version information, allowing for exact app targeting.

4. Store-Based Policies

Parameter: <ProductId>

Value: 9WZDNCRFJ364

→ Apply policies based on Microsoft Store product IDs

This allows policies to be defined using the same IDs that appear in the Microsoft Store, making it easier for administrators who work with store deployments.

To recap to this point:

When a file operation occurs, `StSecGetSecurityDescriptor` analyzes the file path against stored policy templates. For example, if an app tries to access:

`\Documents\Microsoft.Office.Word_8wekyb3d8bbwe\PrivateData\report.docx`

And there's a policy template:

`\Documents\<PackageFamilyName>\PrivateData`, The function recognizes that `Microsoft.Office.Word_8wekyb3d8bbwe` matches the position of `<PackageFamilyName>`.

The driver then calls `StSecpGetParameterValue` with:

Parameter: <PackageFamilyName>

Value: `Microsoft.Office.Word_8wekyb3d8bbwe`

Based on the parameter type, `StSecpGetParameterValue` calls the appropriate SID resolver.

When the dust settles, we are back to `FCpObtainSecurityInfoCallout`:

```
status = StSecGetSecurityDescriptor(chamberPath, securityDescriptor,
chamberId, &ChamberData->ChamberType);

if (securityDescriptor == NULL && chamberId != NULL)
{
    ExFreePoolWithTag(chamberId, POOL_TAG_STsp);
}

if (status < 0)
{
    ...error...
}

if (ChamberData->ChamberId != NULL)
{
    goto FCpObtainSecurityInfoCallout_return;
}
```

But we are not done yet, as you can see, the driver takes into account the possibility of the `ChamberId` not being resolved after the `StSecGetSecurityDescriptor`, and it then resolves to some known values:

```

if ((gFCFlags & EncryptAllFlagBit) != 0)
{
    assignedChamberId = L"{0b7992da-c5e6-41e3-b24f-55419b997a15}";
    goto FCpObtainSecurityInfoCallout_assign_chamberid_and_return;
}

if ((gFCFlags & EncryptMediaFlagBit) == 0)
{
    goto FCpObtainSecurityInfoCallout_return;
}

chamberMatch = RtlPrefixUnicodeString(&gMusicPath, chamberPath, '\\x01');
if (chamberMatch == '\\0')
{
    chamberMatch = RtlPrefixUnicodeString(&gPicturesPath, chamberPath, '\\x01');
    if (chamberMatch != '\\0')
    {
        assignedChamberId = L"PicturesChamber";
        goto FCpObtainSecurityInfoCallout_assign_chamberid_and_return;
    }
    chamberMatch = RtlPrefixUnicodeString(&gVideosPath, chamberPath, '\\x01');
    if (chamberMatch != '\\0')
    {
        assignedChamberId = L"VideosChamber";
        goto FCpObtainSecurityInfoCallout_assign_chamberid_and_return;
    }
}
else
{
    assignedChamberId = L"MusicChamber";
    FCpObtainSecurityInfoCallout_assign_chamberid_and_return:
    ChamberData->ChamberId = assignedChamberId;
}

ChamberData->ChamberType = 1;
FCpObtainSecurityInfoCallout_return:
    ChamberData->Status = status;
}

```

If we fail to find a specific chamber, we first check if the `EncryptAll` flag is set; if it is, a hardcoded GUID is assigned as the `ChamberId`. If the `EncryptAll` flag isn't set, we check if the `EncryptMedia` flag is set. If it is, we compare the path being read to determine if it is indeed one of the known Windows media folders: Pictures, Videos, or Music. Note that in this case, the `ChamberType` is always assigned a value of 1. This is important when we get to the chamber's encryption key derivation process in the post-operation.

How did we get here again?

Right, we were opening a file or something, back to `FCPreCreate`:

```

kernelStackStatus = KeExpandKernelStackAndCalloutEx(
    FCpObtainSecurityInfoCallout,
    &chamberData,
    0x3000,
    0,
    0
);

if (kernelStackStatus < 0)
{
    ...
}
else
{
    securityDescriptor = chamberData.SecurityDescriptor;
    chamberIdStr = chamberData.ChamberId;
    accessMask = chamberData.ChamberType;
    chamberId = chamberData.ChamberId;
}

```

As stated previously, if the stack expansion fails, the same operations from `FCpObtainSecurityInfoCallout` occur inline in `FCPreCreate`.

Next up, the driver uses the security descriptor it now has to check for valid permissions for the current operation. But before we go about doing pesky access checks, we refer to the flag we set earlier in the instance setup:

```

chamberData.Status = STATUS_SUCCESS;
if (volumeContext->VerificationNeeded == FALSE)
{
    FCPreCreate_access_not_modified:
    isAccessModified = FALSE;
}
else
{
    ...
    /* Access check :( */
    chamberData.Status = FCpAccessCheck(Data, securityDescriptor,
    &accessMask);
}

```

`FCpAccessCheck` isn't too exciting; it wraps a call to `SeAccessCheck` and ensures we have the current access (`ACCESS_MASK`) for the current operation. What is important to note is that the call to `SeAccessCheck` can be bypassed entirely when setting the [BypassAccessChecksFlagBit](#) flag:

Inside FCpAccessCheck

```
...
NTSTATUS return_status = STATUS_SUCCESS;
...

if ((gFCFlags & BypassAccessChecksFlagBit) == 0)
{
    isAllowed = SeAccessCheck(
        ...
    );
}

...

return return_status;
```

If FCpAccessCheck succeeds with a positive status, we continue:

```
/* Access check :( */
chamberData.Status = FCpAccessCheck(Data, securityDescriptor,
&accessMask);

if (-1 < chamberData.Status)
{
    goto FCPreCreate_access_not_modified;
}
```

See the previous code snippet for the FCPreCreate_access_not_modified goto.

If not, instead of just aborting the operation, the driver tries to change the original create options and retry the access check:

```
if (chamberData.Status == STATUS_ACCESS_DENIED)
{
    fileCreateOptions = (Data->Iopb->Parameters).Create.Options;
    fileCreateDisposition = fileCreateOptions >> 0x18;
    callbackData = fileCreateDisposition;

    /* Checks if the create disposition is not FILE_OPEN_IF or
FILE_OVERWRITE_IF */
    if ((fileCreateDisposition - 3 & 0xffffffffd) != 0)
    {
        /* Checks if we are creating a file that is not a directory file */
        if (
            fileCreateDisposition == FILE_CREATE &&
            (fileCreateOptions & FILE_DIRECTORY_FILE) != 0
        )
```

```

    {
        chamberData.Status = STATUS_OBJECT_NAME_COLLISION;
        goto FCPreCreate_return_no_post_op;
    }
    goto FCPreCreate_access_not_modified_2;
}

newCreateDisposition = FILE_OVERWRITE;

if (fileCreateDisposition != FILE_OVERWRITE_IF)
{
    newCreateDisposition = FILE_OPEN;
}

/* Rebuild the create options:
 * Keep the original create/open flags and change the disposition */
(Data->Iopb->Parameters).Create.Options =
    newCreateDisposition << 0x18 | fileCreateOptions & 0xffffffff;

/* Try access check again with modified disposition */
chamberData.Status = FcAccessCheck(Data, securityDescriptor,
&accessMask);

if (chamberData.Status != 0)
{
    goto FCPreCreate_access_not_modified_2;
}

isAccessModified = '\x01';

FltSetCallbackDataDirty(Data);
}

```

The driver tries its luck with a new create disposition of FILE_OVERWRITE or FILE_OPEN to make the operation work. If the second access check fails, the entire operation stops, and the post-operation won't be called.

Next, if we were granted access, the driver would call the dreaded FsRtlIsMobileOS once again, one last fateful time:

```

isMobile = FsRtlIsMobileOS();
if (isMobile == '\0')
{
    if (chamberData.ChamberType - 1 < 2)
    {
        (((Data->Iopb->Parameters).Create.SecurityContext)->AccessState->Flags) =
            (((Data->Iopb->Parameters).Create.SecurityContext)->AccessState->Flags) |
                SPECIAL_ENCRYPTED_OPEN;
    }
    if (chamberId != NULL)
    {

```



```

        /* Free chamber ID and return without completion context */
        goto FCPreCreate_cleanup;
    }
}
else if (chamberId != NULL)
{
    /* For mobile OS, create a completion context to pass to post-operation */

    lookasideListEntry =
    ExAllocateFromNPagedLookasideList(&gPre2PostCreateContextList);

    if (lookasideListEntry == NULL)
    {
        chamberData.Status = STATUS_BAD_INITIAL_STACK;

        return_status = FLT_PREOP_SUCCESS_WITH_CALLBACK;
        goto FCPreCreate_cleanup;
    }
    lookasideListEntry->ChamberId = chamberId;
    lookasideListEntry->ChamberType = chamberData.ChamberType;
    lookasideListEntry->IsAccessModified = isAccessModified;
}
*CompletionContext = lookasideListEntry;
goto FCPreCreate_cleanup;

```

If we are on a desktop machine, this is where our journey comes to an end, without any encryption or glory.

“The grey rain-curtain of this world rolls back, and all turns to silver glass.”

On the other hand, on a mobile machine, this is where the fun begins: We set up the completion context with our acquired chamber information.

FCPostCreate

We are back at the IRP_MJ_CREATE post-operation, and without a doubt, on a mobile operating system. After performing some sanity checks, the driver calls `FltIsDirectory` to ensure that the file object is not a directory; if it is, the process does not proceed, and no encryption will occur.

The driver retrieves the volume context (`CUSTOM_FC_VOLUME_CONTEXT`) and allocates a new stream context (`CUSTOM_FC_STREAM_CONTEXT`), which will hold the chamber's encryption key data (`CUSTOM_FC_BCRYPT_KEY_DATA`). After initializing the data of the stream context with default values, the driver calls `FCpEncStreamStart` to start the key acquisition process:

```

status = FltGetVolumeContext(
    FltObjects->Filter,
    FltObjects->Volume,
    (PFLT_CONTEXT*) volumeContext
);

if (-1 < status)

```

```

{
    status = FltAllocateContext(
        gFilterHandle,
        FLT_STREAM_CONTEXT,
        0x28,
        NonPagedPoolNx,
        (PFLT_CONTEXT*) streamContext
    );

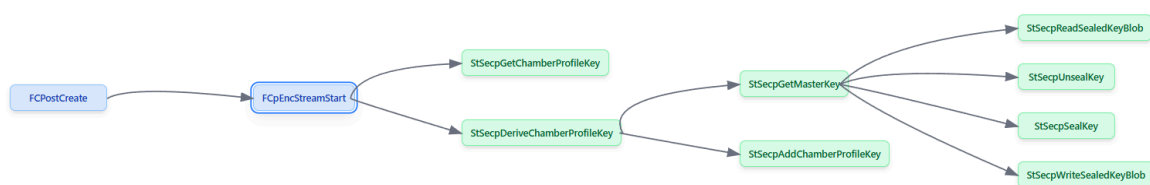
    if (-1 < status)
    {
        streamContext->KeyData.BcryptKeyHandle = NULL;
        streamContext->KeyData.KeyObject = NULL;
        streamContext->KeyData.KeyObjectSize = 0;
        streamContext->ChamberType = 0;

        streamContext->ChamberId = chamberId;
        streamContext->ChamberType = chamberType;
        chamberId = NULL;

        status = FCpEncStreamStart(
            &volumeContext->BcryptAlgHandle,
            streamContext->ChamberId,
            chamberType,
            &streamContext->KeyData
        );
    }
    ...
}

```

The next sequence of events is the following:



The driver will first attempt to call `StSecpGetChamberProfileKey` (the top green box on the left) to check if the chamber's key already exists in its in-memory cache. If it exists, it generates a symmetric key and returns it via an output parameter, which will be assigned to the stream context. If it does not exist in the cache, the driver will call `StSecpDeriveChamberProfileKey` (the bottom green box on the left), which will go all the way down to the TPM to construct the chamber's key.

Let's get into it:

As stated, first try a cache lookup via `StSecpGetChamberProfileKey`:

```

NTSTATUS
FCpEncStreamStart(
    PCUSTOM_FC_BCRYPT_DATA HAlgorithm,
    PWCHAR ChamberId,
    ULONG ChamberType,
    PCUSTOM_FC_BCRYPT_KEY_DATA OutKeyReceiver
)
{
    ...
    chamberProfileKey = ExAllocatePool2(0x40, HAlgorithm->SecretKeySize,
    POOL_TAG_FCCr);

    ...
    profileKeySize = HAlgorithm->SecretKeySize;
    ...

    return_status = StSecpGetChamberProfileKey(ChamberId, ChamberType,
    chamberProfileKey, profileKeySize);

    if (return_status < 0 && return_status == STATUS_OBJECT_NAME_NOT_FOUND)
    {
        pbKeyObject = chamberProfileKey;

        return_status = StSecpDeriveChamberProfileKey(
            ChamberId,
            ChamberType,
            chamberProfileKey,
            profileKeySize
        );

        event = ChamberId;
    }

    ...
}

```

The function takes a chamber identifier and type as input, then derives and outputs a cryptographic key of the specified length. The chamber type determines which of the two derived keys to return - each chamber can have two encryption keys.

```

NTSTATUS
StSecpDeriveChamberProfileKey(
    PWCHAR ChamberId,
    ULONG ChamberType,
    PCHAR OutputProfileKey,
    ULONG ProfileKeyLength
)

```

StSecpDeriveChamberProfileKey starts by first retrieving the *Master Key* by calling StSecpGetMasterKey:

```

NTSTATUS return_status;
ULONG masterKeySize = 0;
PUCHAR masterKey = NULL;

return_status = StSecpGetMasterKey(&masterKey, &masterKeySize);

```

The driver stores an encrypted or *sealed* version of the master key in the [registry](#) as well as **an entirely *unsealed* (decrypted) version in memory in the `g_MasterKey` global variable** - maybe it would have been better to make the font colour red on top of making it bold, but we are not security researchers, so what do we know. The driver reads the sealed key from the registry, unseals it using the [TPM](#), and saves the raw unencrypted key in memory for subsequent use. We won't delve into the code; the readable code is straightforward and essentially does what is stated above. The unreadable parts are the `StSecpUnsealKey` and `StSecpSealKey` functions, which contain calls to the TPM using the API from `tbs.h`. These two functions received the absolute worst end of Ghidra's decompilation, and we did not bother to sort out the mess that it left aside from identifying the TPM calls and their parameters. One helpful thing to note is that the **TPM calls can be bypassed entirely** by setting a special registry flag, which is expanded upon [here](#).

Next, we see the implementation in `StSecpDeriveChamberProfileKey` of the Key Derivation Function (KDF), where it takes a secret key (the master key) and transforms it into multiple different keys. The function uses HMAC (Hash-based Message Authentication Code): First, we initialize a HMAC handle.

```

BCRYPT_HASH_HANDLE ChamberIdHashHandle = NULL;
...

phMasterKeyHash = &ChamberIdHashHandle;

return_status = BCryptCreateHash(
    g_HmacHashProvider,
    phMasterKeyHash,
    pbMasterKeyHashObject,
    g_cbHashObjectLength,
    masterKey,
    masterKeySize,
    0
);

```

This creates an HMAC context using the master key as the HMAC secret. Next, we hash the chamber Id:

```

pbChamberIdInput = ChamberId;
...

return_status = BCryptHashData(ChamberIdHashHandle, pbChamberIdInput,
    chamberIdLength, 0);

```

Why is this important? Each chamber needs unique encryption keys, but we can't just use the chamber Id directly as a key (that would be insecure). Instead, we're creating a key that's mathematically derived from both the master key (secret) and the chamber Id. Now, because we will be generating two keys, instead of doing the two previous operations again, the driver duplicates the result we have up until this point:

```
return_status = BCryptDuplicateHash(  
    ChamberIdHashHandle,  
    &dupChamberIdHashHandle,  
    pbHashObject,  
    g_cbHashObjectLength,  
    0  
);
```

Next, we create the two keys, the “Install” and “Data” keys:

```
return_status = BCryptHashData(ChamberIdHashHandle, L"Install", 0xe, 0);  
  
installKey = ExAllocatePool2(0x40, g_cbHashOutputLength, POOL_TAG_StSn);  
  
return_status = BCryptFinishHash(ChamberIdHashHandle, installKey,  
    g_cbHashOutputLength, 0);  
  
...  
  
return_status = BCryptHashData(dupChamberIdHashHandle, L"Data", 8, 0);  
  
dataKey = ExAllocatePool2(0x40, g_cbHashOutputLength, POOL_TAG_StSn);  
  
return_status = BCryptFinishHash(dupChamberIdHashHandle, dataKey,  
    g_cbHashOutputLength, 0);
```

And of course, this driver never misses a chance to cache whatever it can, so we add both keys to the in-memory cache stored in the g_StSecCacheGenericTable (CUSTOM_FC_STSEC_CACHE_TABLE_ENTRY) global variable:

```
return_status = StSecpAddChamberProfileKey(  
    ChamberId,  
    installKey,  
    dataKey,  
    ProfileKeyLength  
);
```

And finally, the driver selects and returns the key according to the chamber type:

```
if (  
    chosenOutputKey = installKey, ChamberType == 1 ||  
    (chosenOutputKey = dataKey, ChamberType == 2)  
)  
{
```

```

        memcpy(OutputProfileKey, chosenOutputKey, ProfileKeyLength);
    }

    goto StSecpDeriveChamberProfileKey_return_and_cleanup;

```

After deriving our super-specific chamber key, we are back at FCpEncStreamStart at:

```

NTSTATUS
FCpEncStreamStart(
    PCUSTOM_FC_BCRYPT_DATA HAlgorithm,
    PWCHAR ChamberId,
    ULONG ChamberType,
    PCUSTOM_FC_BCRYPT_KEY_DATA OutKeyReceiver
)
...
return_status = StSecpDeriveChamberProfileKey(
    ChamberId,
    ChamberType,
    chamberProfileKey,
    profileKeySize
);

```

Using the chamber key, we create yet another key. This time, we generate the final symmetric AES key for the driver's stream context:

```

return_status = BCryptGenerateSymmetricKey(
    HAlgorithm->BcryptAlgHandle,
    &OutKeyReceiver->BcryptKeyHandle,
    pbKeyObject,
    pbKeyObjectSize,
    chamberProfileKey,
    HAlgorithm->SecretKeySize,
    0
);

```

After successfully generating and deriving about 400 keys, the driver sets the newly created stream context in place by calling FltSetStreamContext, which ultimately ends the create operation.

IRP_MJ_READ - FCPreRead & FCPostRead

The read callbacks are where FileCrypt performs its decryption functionality, but only on mobile systems. Remember from earlier, on non-mobile systems, the driver only registers create callbacks. When applications read data from encrypted files on mobile devices, these functions transparently intercept and decrypt the data before it reaches the application.

FCPreRead

The IRP_MJ_READ operation intercepts file reads before they reach the file system. FCPreRead's primary function is to determine whether the file being read requires decryption and, if so, to establish the necessary context for the post-operation callback.

The function follows a familiar pattern we've seen in other callbacks - it first retrieves the stream context to check if the file is encrypted:

```
FCPreRead(
    PFLT_CALLBACK_DATA CallbackData,
    PCFLT_RELATED_OBJECTS FltObjects,
    PVOID* CompletionContext
)
...
ULONG readLength = (CallbackData->Iopb->Parameters).Read.Length;
...

status = FltGetStreamContext(FltObjects->Instance, FltObjects->FileObject,
streamContext);

if (
    status < 0 ||
    (
        FltObjects->FileObject != NULL &&
        (FltObjects->FileObject->Flags & FO_BYPASS_IO_ENABLED) != 0 ||
        readLength == 0
    )
)
{
    return_status = FLT_PREOP_SUCCESS_NO_CALLBACK;
    goto FCPreRead_cleanup_and_return;
}
```

```
lookasideListEntry =
ExAllocateFromNPagedLookasideList(&gPre2PostIoContextList);

...

status = FltSetFsZeroingOffsetRequired(CallbackData);
if (-1 < status)
{
    eventParam = CallbackData;
```

```

FtlSetCallbackDataDirty(CallbackData);

lookasideListEntry->VolumeContext = volumeContext;
lookasideListEntry->StreamContext = streamContext;
*CompletionContext = lookasideListEntry;

return_status = FLT_PREOP_SUCCESS_WITH_CALLBACK;

goto FCPreRead_cleanup_and_return;
}

```

FCPreRead doesn't modify the I/O operation itself. Instead, it sets up all the necessary data to be used later in the post-operation for the actual decryption. After confirming that decryption is needed, a completion context of type `CUSTOM_FC_READ_CONTEXT` is saved on a newly allocated lookaside list and passed on to the post-operation. An interesting security feature to notice in the code above is the call to `FtlSetFsZeroingOffsetRequired`. This routine sets a flag that ensures any padding beyond the actual file content is zeroed out during the read operation, preventing potential data leakage from previous file contents that may still exist in unallocated disk sectors.

FCPostRead

FCPostRead is where the actual decryption happens. The function first checks if an immediate decryption is possible, or whether it needs to be relegated to a work item from a nonpaged pool:

```

FLT_POSTOP_CALLBACK_STATUS
FCPostRead(
    PFLT_CALLBACK_DATA CallbackData,
    PCFLT_RELATED_OBJECTS FtlObjects,
    PCUSTOM_FC_READ_CONTEXT CompletionContext,
    FLT_POST_OPERATION_FLAGS Flags
)
{
    ...
    KIRQL currentIrql;
    PFLT_GENERIC_WORKITEM fltWorkItem = NULL;
    PCUSTOM_FC_DECRYPT_PARAMS fcDecryptParams = NULL;
    PCUSTOM_FC_DECRYPT_PARAMS contexts = CompletionContext;
    ...

    if (
        currentIrql < DISPATCH_LEVEL ||
        CallbackData->IoStatus.Information < 0x20001 &&
        (
            CallbackData->Iopb->Parameters.Others.Argument5 != NULL ||
            (CallbackData->Flags & FLTFL_CALLBACK_DATA_SYSTEM_BUFFER) != 0
        )
    )

```



```

{
    ...
    FCDecryptWorker(NULL, FltObjects->Instance, contexts);
    ...
}
else
{
    fltWorkItem = FltAllocateGenericWorkItem();
    if (fltWorkItem != NULL)
    {
        ...
        ioStatus = FltQueueGenericWorkItem(
            fltWorkItem,
            FltObjects->Instance,
            FCDecryptWorker,
            DelayedWorkQueue,
            fcDecryptParams
        );
        ...
    }
}
}

```

The driver is looking for small read buffers that it can decrypt immediately; moreover, the `FLTFL_CALLBACK_DATA_SYSTEM_BUFFER` flag indicates that the buffer is from a non-paged pool and has already been locked down for safe access. If not, it will relegate the decryption to a work item that it queues.

We found the 131,073-byte threshold particularly interesting - it's exactly 128 KB + 1 byte, suggesting that Microsoft identified this as an optimal balance point between immediate processing and system responsiveness.

For asynchronous processing, the function allocates a work item and queues it for later execution, returning `FLT_POSTOP_MORE_PROCESSING_REQUIRED` to inform the filter manager that processing will be completed later.

Whether called synchronously or asynchronously, `FCDecryptWorker` is called (directly or indirectly). The function first ensures it has access to the encrypted buffer. If no MDL is available, it attempts to lock the user buffer.

```

VOID
FCDecryptWorker(
    PFLT_GENERIC_WORKITEM WorkItem,
    PFLT_INSTANCE Instance,
    CUSTOM_FC_DECRYPT_PARAMS* Params
)
{
    NTSTATUS status;
    PVOID ppvVar4;
    ULONG zeroingOffset = 0;
    PCUSTOM_FC_READ_CONTEXT completionContext = Params->CompletionContext;

```

```

PFLT_CALLBACK_DATA callbackData = Params->CallbackData;
PFLT_IO_PARAMETER_BLOCK iopb = callbackData->Iopb;
PMDL mdlAddress = (iopb->Parameters).Read.MdlAddress;
ULONG sectorSize;
PEVENT_DESCRIPTOR eventDescriptor;
PVOID event;
PFLT_CALLBACK_DATA event2;
PVOID event3 = &Params->CallbackData;

if (mdlAddress == NULL)
{
    if ((callbackData->Flags & FLTFL_CALLBACK_DATA_SYSTEM_BUFFER) == 0)
    {
        event2 = callbackData;
        status = FltLockUserBuffer(callbackData);

        if (status < 0)
        {
            if ((Microsoft_Windows_FileCryptEnableBits & 1) != 0)
            {
                McTemplateK0d_EtwWriteTransfer(event2,
&LockUserBufferFailure, event3, status);
            }
            goto FCDecryptWorker_cleanup_and_return;
        }

        mdlAddress = (iopb->Parameters).Read.MdlAddress;
        goto LAB_1c0002288;
    }
}
...

```

The actual decryption happens in `FCpEncDecrypt`, which processes the data in sector-sized chunks using the AES-CBC algorithm and the file's chamber-specific encryption key. The zeroing offset ensures that any padding beyond the actual file content gets properly zeroed for security.

Its first critical task involves ensuring access to a usable virtual address for the encrypted data buffer. When no MDL (Memory Descriptor List) is initially available, `FCDecryptWorker` distinguishes between system and user buffers by checking the `FLTFL_CALLBACK_DATA_SYSTEM_BUFFER` flag.

For user buffers, it calls `FltLockUserBuffer` to create an MDL that maps the user-space buffer into kernel-addressable memory. This operation can fail if the buffer cannot be locked, triggering ETW event logging and causing the function to terminate early. Kernel buffers are already mapped with an MDL to kernel-addressable memory, so we don't need to perform another action for them.

Once an MDL exists, the function must obtain a virtual address for the physical memory pages. It examines the MDL flags to determine if the memory is already mapped or originates from the non-paged pool.

If neither the MDL_MAPPED_TO_SYSTEM_VA nor MDL_SOURCE_IS_NONPAGED_POOL flags are set, it calls MmMapLockedPagesSpecifyCache to establish a system virtual address mapping using cached memory with a default MDL protection, plus additional flags. For already-mapped memory, it simply uses the existing MappedSystemVa pointer.

Finally, FCpEncDecrypt is simply a wrapper function for the BCryptDecrypt function, which is part of the bcrypt.h. FCpEncDecrypt, which receives the volume's BCrypt algorithm handle and the stream's key data, calls BCryptDecrypt once per sector to decrypt each sector individually. The size of the buffer to decrypt is the sector size rounded up to the next sector.

IRP_MJ_WRITE - FCPreWrite & FCPostWrite

The write callbacks are where FileCrypt performs its actual encryption functionality. When applications write data to encrypted files, these callbacks transparently intercept and encrypt the data before it reaches the disk. Just like the read callback, it is applicable only for mobile systems.

FCPreWrite

The IRP_MJ_WRITE operation intercepts write operations before they reach the file system. The FCPreWrite function begins by retrieving the stream context to determine if the file requires encryption. If no stream context exists, the file isn't encrypted, and the driver returns FLT_PREOP_SUCCESS_NO_CALLBACK.

```
FLT_PREOP_CALLBACK_STATUS
FCPreWrite(
    PFLT_CALLBACK_DATA CallbackData,
    PCFLT_RELATED_OBJECTS FltObjects,
    PVOID* CompletionContext
)
{
    NTSTATUS ioStatus;
    PVOID setter;
    PVOID generalPtr;
    FLT_PREOP_CALLBACK_STATUS return_status = FLT_PREOP_SUCCESS_NO_CALLBACK;
    PCUSTOM_FC_STREAM_CONTEXT streamContext = NULL;
    ULONG writeLength = (ioqb->Parameters).Write.Length;
    ...

    setter = &streamContext;
    generalPtr = FltObjects->Instance;
    ioStatus = FltGetStreamContext(generalPtr, FltObjects->FileObject, setter);

    if (
        ioStatus < 0 ||
        FltObjects->FileObject != NULL &&
        (FltObjects->FileObject->Flags & FO_BYPASS_IO_ENABLED) != 0
    )
}
```

```

{
    return_status = FLT_PREOP_SUCCESS_NO_CALLBACK;
    goto FCPreWrite_cleanup_and_return;
}

if (writeLength == 0)
{
    goto FCPreWrite_cleanup_and_return;
}

```

After confirming encryption is needed, the function retrieves the volume context to access its sector size and BCrypt algorithm information. This leads to an interesting memory allocation decision in the driver.

The driver implements a two-tier allocation strategy based on write size. For writes smaller than 64 KB, it allocates buffers from a pre-initialized lookaside list (the `gShadowBufferList` global variable). For larger writes, it allocates directly from the nonpaged pool using the `ExAllocatePool2` routine. The allocation type gets stored in a flag (1 for lookaside list, 2 for pool) that will be passed to the post-operation.

```

totalSizeToEncrypt = (writeLength - 1) + volumeContext->SectorSize &
-volumeContext->SectorSize;

if (totalSizeToEncrypt < 0x10001)
{
    shadowBufferPtr = &gShadowBufferList;
    ciphertext = ExAllocateFromNPagedLookasideList(&gShadowBufferList);
    allocationType = '\x01';
}
else
{
    ciphertext = ExAllocatePool2(0x40, totalSizeToEncrypt, POOL_TAG_FCsb);
    allocationType = '\x02';
}

```

This alignment of the length is required for block-cipher encryption, as you cannot encrypt partial sectors. Hence, the calculation at the `totalSizeToEncrypt` assignment rounds up to the next sector boundary.

Once a buffer for the ciphertext is allocated, `FCPreWrite` creates an MDL (Memory Descriptor List) to describe it and then performs the written buffer replacement. The function modifies the I/O request to point to the encrypted buffer instead of the original plaintext buffer:

```

mdl = IoAllocateMdl(ciphertext, totalSizeToEncrypt, 0, 0, 0);
allocationTypeCopy = allocationType;

if (mdl == NULL)
{
    goto FCPreWrite_set_allocation_failure_status;
}

```

```

}

MmBuildMdlForNonPagedPool(mdl);
MmMdlPageContentsState(mdl, 1);
generalPtr = (ioqb->Parameters).Others.Argument5;

```

The driver then encrypts the original write buffer plaintext and replaces the write buffer with the ciphertext:

```

plaintext = (PUCHAR)(ioqb->Parameters).Write.MdlAddress;
FCPreWrite_encrypt:
setter = plaintext;
ioStatus = FCpEncEncrypt(
    &volumeContext->BcryptAlgHandle,
    &streamContext->KeyData,
    plaintext,
    ciphertext,
    totalSizeToEncrypt,
    ioqb->Parameters.Write.WriteBuffer
);
generalPtr = &gPre2PostIoContextList;
lookasideEntry =
ExAllocateFromNPagedLookasideList(&gPre2PostIoContextList);

```

The critical step comes next - replacing the original I/O parameters with the encrypted buffer:

```

generalPtr = &gPre2PostIoContextList;
lookasideEntry =
ExAllocateFromNPagedLookasideList(&gPre2PostIoContextList);

if (lookasideEntry != NULL)
{
    (ioqb->Parameters).Write.MdlAddress = ciphertext;
    (ioqb->Parameters).Others.Argument5 = mdl;
    generalPtr = (NPAGED_LOOKASIDE_LIST*)CallbackData;
    FltSetCallbackDataDirty(CallbackData);
    lookasideEntry->Ciphertext = ciphertext;
    lookasideEntry->AllocationType = allocationType;
    lookasideEntry->VolumeContext = volumeContext;
    lookasideEntry->StreamContext = streamContext;
    *CompletionContext = lookasideEntry;
    return_status = FLT_PREOP_SUCCESS_WITH_CALLBACK;
    goto FCPreWrite_cleanup_and_return;
}
goto FCPreWrite_set_allocation_failure_status;
}

```

What we also see is the passing of the ciphertext, allocation type, and contexts to the post operation.

FCPostWrite

FCPostWrite is significantly simpler than its pre-operation counterpart. Its only responsibility is cleanup and resource management after the write operation completes. The function unpacks all the data passed on to it from the pre-operation and simply invokes multiple cleanup calls.

```
FLT_POSTOP_CALLBACK_STATUS
FCPostWrite(
    PFLT_CALLBACK_DATA CallbackData,
    PCFLT_RELATED_OBJECTS RelatedObjects,
    PCUSTOM_FC_WRITE_CONTEXT CompletionContext
)
{
    FltReleaseContext(CompletionContext->VolumeContext);
    FltReleaseContext(CompletionContext->StreamContext);

    FCFreeShadowBuffer(
        CompletionContext->StreamContext,
        CompletionContext->Ciphertext,
        CompletionContext->AllocationType
    );

    ExFreeToNPagedLookasideList(&gPre2PostIoContextList, CompletionContext);

    return FLT_POSTOP_FINISHED_PROCESSING;
}
```

FileCrypt Reimagined - A complete C Rewrite

When we came near the end of the driver's static analysis and reverse engineering, we decided we wanted to take a step further and try to see if we could build the driver from "source". The source, obviously, isn't available. In our Ghidra project, though, we have reached the point where *every* single function inside the driver has undergone full/partial annotation (Most have received full annotations, renames, and proper type assignments). Thus, we decided to bring our decompiled code to life.

Although the option to export the entire project or individual decompiled files exists in Ghidra, it doesn't do much beyond dumping the code that can be viewed in the decompiler window into a file. This leaves us with a lot of code that isn't actually valid C, for example:

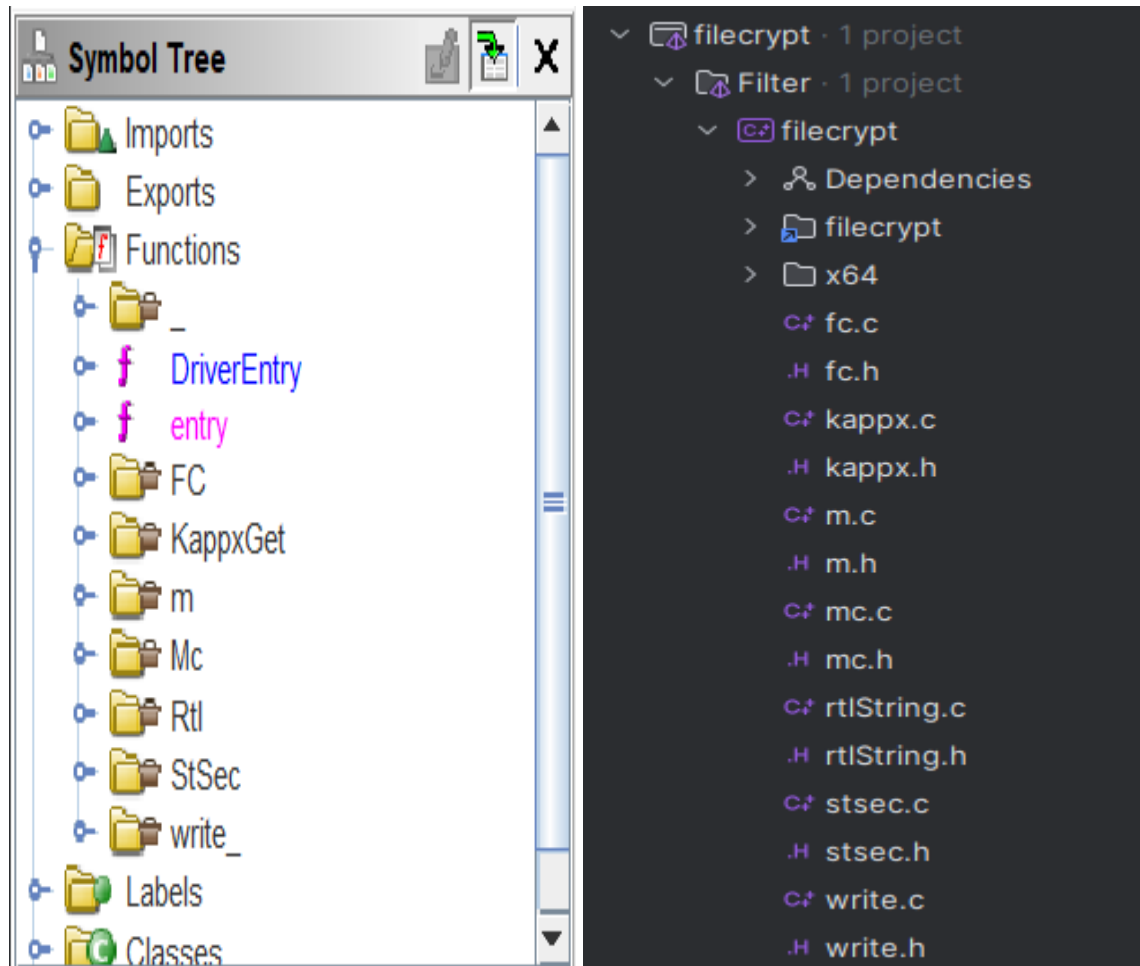
```
chamberPath = CONCAT88(fullPathBuffer, chamberPath._0_8_);
```

All of these anomalies and fillers must be dealt with manually. Each case must be examined individually to minimize errors caused by manual code editing.

We left almost all (I want to say all, but I'm afraid my memory fails me) of the original, decompiled lines in a comment above the edited line. Moreover, each function can be

compared with its original decompiled version by examining it directly in Ghidra using our exported [Ghidra project](#).

The solution contains seven pairs of .c and .h files, each file contains functions that have the file name as their prefix, for example, in **stsec.c** you may find **StSecpDeriveChamberProfileKey**. This brilliant idea came from Ghidra's automatic function groupings:



The entire “source” code can be found in the [GitHub](#) project.

The project does build and yield a proper .sys file, but we take exactly 0x0 responsibility, accountability, and fancier legal terms for the results of trying to run it.

Debugging Windows drivers

When setting up our debugging environment, we started by creating a Windows 11 virtual machine in VMware Workstation. NASA engineers of the 60s couldn't comprehend the sheer amount of RAM things like web browsers would need to display this poorly written article on your screen, let alone draw [React Native Windows start menus](#). The Apollo 11 mission's guidance computer had 4KB of RAM, but we decided to give our VM 12 GB of RAM with the

hope that the machine's File Explorer would take our modest offering kindly. But more importantly, we needed to configure a serial port for the debugger connection.

In the machine's settings in VMware, we added a serial port configured as a named pipe. We set it to `\\.\pipe\windbg` on the host side, configured as a server that waits for an application to connect. This creates the communication channel between our host debugging machine and the target VM.

```
bcdedit /debug on
```

```
bcdedit /dbgsettings serial debugport:1 baudrate:115200
```

The debug settings configure Windows to listen for a kernel debugger on COM1 at 115200 baud, which corresponds to our VMware serial pipe.

On our host machine, we installed the Windows SDK to get WinDbg. We configured it with matching settings to our virtual machine.

Now, with this setup, we can connect to our virtual machine using Windbg and debug the kernel, but improvements can still be made. When debugging a driver with Windbg, it is hard to make connections between the assembly and the C code of the thing we are trying to debug - the filecrypt driver. Fortunately, a plugin called [ret-sync](#) enables us to synchronise Windbg with Ghidra. We followed the installation guide on GitHub and went through this handy [video](#) (Toda le YM). By the end, we were able to debug the driver and follow its execution in Ghidra.

Ghidra Server setup

To work on a reverse engineering project in a team, we needed to figure out a way to share our work with each other. The first thing we tried was using the tool with which we are most familiar in our daily work - Git. This turned out to be a problem, as the files generated by Ghidra do not work at all with a source control like Git, so we had to find an alternative.

Luckily for us, Ghidra comes bundled with a runnable service called Ghidra Server, which allows multiple people working on the same project to synchronize their work. Unlike Git, Ghidra Server does not have an equivalent to GitHub, so we had to host our own instance of the Ghidra Server.

Before setting up the server, we needed a place to host it. We explored several cloud platforms and ultimately chose [Kamatera](#) because it offered affordable Ubuntu VMs with sufficient resources to run Ghidra Server, and came pre-installed with MSTSC remote desktop interface for ease of use.

Now all we have to do is set up the server:

1. First, git clone this [bad boy](#) on your server

2. Make sure you've got pesky Java 17 installed (`sudo apt install openjdk-17-jdk`)
3. Build it - `./gradlew buildGhidra` and move the finished build to `/opt/ghidra`. Create a dedicated user for the server - `sudo useradd -r ghidra-server`
4. Hop into the server directory, and set up your admin account with `sudo -u ghidra-server ./svrAdmin -add <your_admin_name>`
5. To run it automatically, create a simple systemd service file that points to the `ghidraSvr` script. Then, enable it with `sudo systemctl enable ghidra-server && sudo systemctl start ghidra-server`. Don't forget to open port 13100 in your firewall. Once it's running, you can add team members with `./svrAdmin -add <username>`
6. Create shared projects with `./svrAdmin -create <project_name>`
7. Now your teammates can connect through their personal Ghidra by clicking on the "New Shared Project" option and typing your server's IP address

Closure

We went into this project completely blind and uninformed about the magic and wonder of Windows driver programming and the hair pulling frustration of reverse engineering. But finally, when all is said and done this project was really fun and informative. Even though the driver is basically useless, hopefully another glimpse into the wonderful systems of Microsoft might provide further enlightenment to the Windows community.

"So long, and thanks for all the fish."